AquaSense

Monitor Inteligente de Umidade do Solo · Documentação Técnica

Grupo 7 · Inovatech 2026 · Versão 1.1

Sumário

I. Cabeçalho e Identificação · II. Escopo Arquitetural · III. Lista de Materiais (BOM) · IV. Engenharia de Memória · V. Topologia de Conexão · VI. Manual de Deploy

I. Cabeçalho e Identificação

Sistema	AquaSense: Monitor Inteligente de Umidade do Solo
Nicho temático	Agricultura e irrigação (vertical Umidade do Solo)
Versão	1.1, firmware da placa ESP32 apresentado no Inovatech (11/11/2026). A versão 1.0 é a planta de execução em C, homologada em 03/11/2026.
Integrantes	Danilo Martinelli Bueno Veiga Manoel Florenço da Silva Júnior Rian Godinho Caixeta Oliveira Martins Yan Nascimento Sousa
Repositório	https://github.com/Danilexxx/aquasense-inovatech
Demonstração	https://danilexxx.github.io/aquasense-inovatech/

II. Escopo Arquitetural

Problema físico mapeado

Solo. A irrigação feita sem medição leva a dois erros. Com pouca água, o solo chega ao ponto de murcha e a raiz deixa de absorver. Com água demais, o solo passa da capacidade de campo, a água escorre e a raiz apodrece.

O AquaSense mede a umidade do solo de forma contínua, classifica o estado do solo e decide sozinho quando a irrigação deve ligar e desligar.

Grandeza medida

A sonda resistiva fica enterrada no solo. O módulo do sensor converte a resistência entre as duas hastes em uma tensão (saída **AO**), que o ADC de 12 bits da ESP32 transforma em um valor de **0 a 4095**. Solo mais úmido conduz melhor e gera um valor menor.

O valor bruto é convertido em **percentual de umidade** por interpolação linear em dois trechos. A calibração foi feita pelo grupo com 3 potes da mesma terra vermelha (argilosa), em 11/11/2026:

Ponto de calibração	ADC	Umidade
Solo seco	4095	0%
Solo ideal	2411	50%
Solo encharcado	1277	100%

```
if (adc >= ADC_IDEAL) pct = (ADC_SECO - adc) * 50.0f / (ADC_SECO - ADC_IDEAL);          /* 0 a 50%
*/
else                  pct = 50.0f + (ADC_IDEAL - adc) * 50.0f / (ADC_IDEAL - ADC_MOLHADO); /* 50 a
100% */
```

Métrica de acionamento lógico

A regra usa **histerese**, com dois limites:

Condição	Ação
Umidade abaixo de **30%** com a bomba desligada	Liga a bomba
Umidade acima de **60%** com a bomba ligada	Desliga a bomba
Entre 30% e 60%	Mantém o estado atual

Com um limite único, o ruído natural da leitura faria a bomba ligar e desligar várias vezes por segundo. A faixa entre 30% e 60% evita esse liga-desliga e faz a irrigação trabalhar em ciclos longos.

Os mesmos limites classificam o solo: **SECO** (abaixo de 30%), **IDEAL** (30% a 60%) e **ENCHARCADO** (acima de 60%).

A montagem apresentada não tem relé nem bomba. O acionamento é executado pelo firmware: a saída **GPIO26** (reservada ao relé) muda de nível, o LED embutido da placa (**GPIO2**) acende e o evento é registrado na pilha e informado no monitor serial e no painel. Para irrigar de verdade, basta ligar um módulo relé no GPIO26, sem alterar o código.

Esteira de processamento

```
Sonda + módulo (AO)
   |  média de 16 amostras do ADC (GPIO35)
   v
struct Leitura  --enfileirar-->  FILA (FIFO)
                                    |  a cada 3 leituras: desenfileirar
                                    v
                        Regra de acionamento (histerese)
                          |                       |
              muda o estado da bomba        GPIO26 + LED (GPIO2)
                          v
                PILHA (LIFO) de eventos
```

III. Lista de Materiais (BOM)

Qtd	Item	Especificação	Função
1	ESP32 DevKit	38 pinos, ADC de 12 bits, conversor USB-serial CP210x, conector micro-USB	Microcontrolador: lê o sensor e executa a esteira FIFO/LIFO
1	Sonda de umidade do solo	Tipo resistiva, duas hastes metálicas, cabo de 2 vias	Elemento sensor, enterrado no solo
1	Módulo driver do sensor	Comparador LM393, saídas AO (analógica) e DO (digital), potenciômetro de sensibilidade, alimentação 3,3 a 5 V	Converte a resistência da sonda em tensão
1	Protoboard		Fixação da ESP32 e interligação
3	Jumpers	Vermelho (energia), preto (terra), verde (sinal)	Ligação entre o módulo e a ESP32
1	Cabo micro-USB	Cabo de dados	Alimentação, gravação e comunicação serial
1	Notebook com Google Chrome	Web Serial API	Painel de monitoramento (`medidor.html`)
3	Potes de terra	Mesma terra vermelha: seca, ideal e encharcada	Calibração

Custo: sensor de umidade (sonda e módulo), R$ 55,00.

IV. Engenharia de Memória

1. Mapeamento das variáveis nas `struct`

Na ESP32 (32 bits), `int`, `float`, `enum` e ponteiros ocupam 4 bytes cada.

`Leitura` (16 bytes): a unidade de dado que percorre a esteira.

Campo	Tipo	Bytes	Conteúdo
`id`	`int`	4	Número sequencial da leitura
`adc_bruto`	`int`	4	Valor do ADC (0 a 4095)
`umidade`	`float`	4	Umidade convertida (0 a 100%)
`status`	`StatusSolo`	4	`SOLO_SECO`, `SOLO_IDEAL` ou `SOLO_ENCHARCADO`

`No` (20 bytes): nó da lista encadeada, alocado no heap com `malloc`.

Campo	Tipo	Bytes	Conteúdo
`dado`	`Leitura`	16	A leitura armazenada
`proximo`	`struct No *`	4	Endereço do próximo nó (`NULL` no último)

`Fila` (12 bytes): descritor da fila FIFO.

Campo	Tipo	Bytes	Conteúdo
`inicio`	`No *`	4	Primeiro nó, de onde se remove
`fim`	`No *`	4	Último nó, onde se insere
`tamanho`	`int`	4	Quantidade de nós (limite `CAP_FILA` = 10)

`Evento` (12 bytes): registro de acionamento da bomba.

Campo	Tipo	Bytes	Conteúdo
`id_leitura`	`int`	4	Leitura que causou o evento
`umidade`	`float`	4	Umidade no momento do evento
`acao`	`AcaoBomba`	4	`BOMBA_LIGOU` ou `BOMBA_DESLIGOU`

`Pilha` (64 bytes): histórico LIFO em vetor estático.

Campo	Tipo	Bytes	Conteúdo
`itens`	`Evento[5]`	60	Vetor de 5 eventos (`CAP_PILHA`)
`topo`	`int`	4	Índice do topo (`-1` indica pilha vazia)

`Estatisticas` (16 bytes): `total` (`int`), `soma`, `minimo` e `maximo` (`float`).

Variáveis globais de controle: `bomba_ligada`, `proximo_id`, `total_malloc`, `total_free` (auditoria de memória) e `ultima_leitura` (temporização com `millis()`).

Memória dinâmica no pior caso: 10 nós de 20 bytes, ou seja, **200 bytes**. A pilha ocupa 64 bytes fixos.

2. Motores da esteira FIFO e LIFO

A **fila** guarda as leituras porque o sensor produz dados em ordem temporal e a regra precisa avaliá-los na ordem de chegada. Ela separa quem produz (uma leitura por segundo) de quem consome (processamento a cada 3 leituras). Foi implementada como lista encadeada, com alocação dinâmica, e o ponteiro `fim` garante inserção em tempo constante.

A **pilha** guarda o histórico da bomba porque o evento que importa é o mais recente, e ele fica sempre no topo. Como o tamanho máximo é conhecido, ela usa um vetor estático, sem `malloc`.

Enfileiramento (`enfileirar`, O(1))

```
int enfileirar(Fila *f, Leitura dado) {
    if (fila_cheia(f)) return 0;                  /* overflow bloqueado antes de alocar */
    No *novo = (No *) malloc(sizeof(No));
    if (novo == NULL) return 0;                   /* falha de alocação */
    total_malloc++;
    novo->dado = dado;
    novo->proximo = NULL;
    if (fila_vazia(f)) f->inicio = novo;
    else               f->fim->proximo = novo;    /* liga direto no último nó */
    f->fim = novo;
    f->tamanho++;
    return 1;
}
```

Remoção da fila (`desenfileirar`, O(1))

```
int desenfileirar(Fila *f, Leitura *saida) {
    if (fila_vazia(f)) return 0;                  /* underflow bloqueado */
    No *removido = f->inicio;
    *saida = removido->dado;                      /* copia antes de liberar */
    f->inicio = removido->proximo;
    if (f->inicio == NULL) f->fim = NULL;
    free(removido);
    total_free++;
    f->tamanho--;
    return 1;
}
```

Empilhamento (`empilhar`, O(1))

```
int empilhar(Pilha *p, Evento e) {
    if (pilha_cheia(p)) return 0;                 /* topo == CAP_PILHA - 1 */
    p->topo++;
    p->itens[p->topo] = e;
    return 1;
}
```

Remoção da pilha (`desempilhar`, O(1))

```
int desempilhar(Pilha *p, Evento *saida) {
    if (pilha_vazia(p)) return 0;                 /* topo == -1 */
    *saida = p->itens[p->topo];
    p->topo--;
    return 1;
}
```

No firmware, cada recusa também imprime uma mensagem no monitor serial (`[OVERFLOW BLOQUEADO]` ou `[UNDERFLOW BLOQUEADO]`).

Complexidade

Operação	Ordem	Motivo
`enfileirar`	O(1)	Insere pelo ponteiro `fim`, sem percorrer a lista
`desenfileirar`	O(1)	Remove pelo ponteiro `inicio`
`empilhar`, `desempilhar`	O(1)	Acesso direto pelo índice `topo`
`fila_vazia`, `fila_cheia`, `pilha_vazia`, `pilha_cheia`	O(1)	Uma comparação
`processar_fila`	O(n), n ≤ 10	Visita cada leitura pendente uma vez
`exibir_fila`, `exibir_pilha`	O(n)	Percorrem a estrutura para imprimir

3. Mitigação de erros físicos

Risco	Proteção no código
Overflow da fila	`fila_cheia()` é testada antes do `malloc`. Com 10 nós a leitura é recusada, e o heap usado nunca passa de 200 bytes.
Falha de alocação	O retorno do `malloc` é comparado com `NULL` antes de qualquer uso.
Underflow da fila	`fila_vazia()` é testada antes de acessar `inicio`, o que impede desreferenciar `NULL`.
Overflow da pilha	`pilha_cheia()` (`topo == 4`) é testada antes de escrever. Não há escrita fora de `itens[0..4]`.
Underflow da pilha	`pilha_vazia()` (`topo == -1`) é testada antes de ler. Nunca se lê `itens[-1]`.
Estouro da pilha de chamadas	Não há recursão. A pilha de eventos é global e de tamanho fixo, então o uso da pilha do processador é constante.
Vazamento de memória	Cada nó criado em `enfileirar` é liberado em `desenfileirar`, e `processar_fila` esvazia a fila a cada ciclo. Os contadores `total_malloc` e `total_free` comprovam o balanço em execução (comando `e`).
Ponteiro pendente	Quando a fila esvazia, `fim` volta a `NULL` junto com `inicio`.
Laço travado	A temporização usa `millis()` em vez de `delay()`, então o `loop()` continua atendendo comandos.

As funções de inserção e remoção retornam 1 (sucesso) ou 0 (recusa), e o chamador só avança o contador de leituras quando a operação deu certo.

Verificação ao vivo pelo monitor serial: `f` mostra a fila, `p` mostra a pilha, `d` desempilha (inclusive provocando o underflow) e `e` mostra estatísticas e o balanço de `malloc` e `free`.

V. Topologia de Conexão

Módulo do sensor	ESP32	Jumper	Função
VCC	3V3	Vermelho	Alimentação a 3,3 V
GND	GND	Preto	Terra
AO	**GPIO35**	Verde	Sinal analógico de umidade (ADC1, 12 bits, atenuação de 11 dB)
DO	não conectado		Saída digital de limiar, não utilizada

Ligação	Detalhe
Sonda → módulo	Cabo de 2 vias no conector da ponta do módulo, sem polaridade
ESP32 → notebook	Cabo micro-USB: energia, gravação e serial a 115200 baud
GPIO26	Saída reservada ao relé da bomba, sem componente ligado nesta montagem
GPIO2	LED embutido da placa, aceso enquanto a bomba está ligada

Notas:

- O GPIO35 é do ADC1, que continua disponível com o Wi-Fi ligado, e é um pino somente de entrada.
- O módulo é alimentado em 3,3 V para que a saída AO não passe da tensão máxima do ADC da ESP32.
- O projeto previa o GPIO34. Na placa usada esse pino ficou fixo em cerca de 690, mesmo sem nada ligado, e o sinal foi transferido para o GPIO35. Na simulação do Wokwi o potenciômetro continua no GPIO34.

VI. Manual de Deploy

Dependências

Dependência	Versão	Uso
`<stdlib.h>`	biblioteca padrão C	`malloc`, `free`, `NULL`
Núcleo Arduino para ESP32 (`esp32:esp32`)	3.3.4	`Serial`, `analogRead`, `analogReadResolution`, `analogSetPinAttenuation`, `pinMode`, `digitalWrite`, `millis`

Não há bibliotecas de terceiros. A planta de execução em C (`codigo-c/aquasense.c`) usa apenas `<stdio.h>`, `<stdlib.h>` e `<time.h>`.

Comunicação serial

Parâmetro	Valor
Baud rate	**115200**
Formato	8N1 (8 bits de dados, sem paridade, 1 bit de parada)
Fim de linha	`\n`

Gravação na placa

1. Instalar o Arduino IDE 2 e, no Gerenciador de Placas, o pacote **esp32** da Espressif (3.3.4).
2. Abrir `aquasense_esp32/aquasense_esp32.ino`.
3. Selecionar a placa **ESP32 Dev Module** e a porta COM da placa.
4. Clicar em **Carregar**. Se a gravação parar em `Connecting...`, segurar **BOOT**, apertar e soltar **EN**, e soltar **BOOT**.
5. Ao final, apertar **EN** para reiniciar a placa.
6. Abrir o Monitor Serial a **115200**. Comandos: `f` (fila), `p` (pilha), `d` (desempilhar), `e` (estatísticas e memória).

Pela linha de comando:

```
arduino-cli compile --fqbn esp32:esp32:esp32 aquasense_esp32
arduino-cli upload  --fqbn esp32:esp32:esp32 -p COM4 aquasense_esp32
```

Painel de monitoramento

Abrir `medidor.html` no Google Chrome, clicar em conectar e escolher a porta da ESP32. O painel usa a Web Serial API, sem servidor e sem internet. A placa envia 10 linhas por segundo no formato:

```
DADO;id=12;adc=2870;umid=42.5;bomba=0
```

Enquanto o painel estiver aberto, o Chrome ocupa a porta serial. Para gravar a placa ou usar o Monitor Serial, feche o painel antes.

Execução sem hardware

- Planta de execução em C: `gcc -Wall -Wextra -o aquasense codigo-c/aquasense.c` e depois `./aquasense`.
- Simulação no Wokwi: instruções em `wokwi/LEIAME.md`. Um potenciômetro substitui o sensor.

Constantes do firmware

Constante	Valor	Significado
`LIMITE_SECO`	30.0	Liga a bomba abaixo deste percentual
`LIMITE_ENCHARCADO`	60.0	Desliga a bomba acima deste percentual
`CAP_FILA`	10	Capacidade da fila
`CAP_PILHA`	5	Capacidade da pilha
`ADC_SECO`, `ADC_IDEAL`, `ADC_MOLHADO`	4095, 2411, 1277	Pontos de calibração
`INTERVALO_LEITURA`	1000 ms	Leitura que entra na fila (1 por segundo)
`INTERVALO_ENVIO`	100 ms	Envio ao painel (10 por segundo)
`PROCESSAR_A_CADA`	3	A fila é processada a cada 3 leituras
`AMOSTRAS_POR_LEITURA`	16	Média de amostras do ADC

Estrutura do repositório

Caminho	Conteúdo
`aquasense_esp32/aquasense_esp32.ino`	Firmware da placa (v1.1)
`medidor.html`	Painel de monitoramento via Web Serial
`codigo-c/aquasense.c`	Planta de execução em C (v1.0)
`wokwi/`	Simulação no Wokwi
`index.html`	Site de demonstração (GitHub Pages)
`docs/`	Esta documentação em PDF